المحاضرة الرابعة: الصيغ التقاربية Omega (Ω) وTheta (Θ)

في الجزء السابق من محاضرتنا، ركزنا على صيغة **Big-O (O)**، التي تصف **الحد الأعلى (Upper Bound)** لأداء الخوارزمية، أو بعبارة أخرى، تصف الحالة الأسوأ. لكن ماذا عن الحالة الأفضل؟ وماذا لو أردنا وصفاً أكثر دقة يجمع بين الحالتين؟ هنا يأتي دور صيغتي Omega وTheta.

1. صيغة - Omega (Ω) الحد الأدنى(Lower Bound)

ما هي صيغة Omega ؟

صيغة **Omega (Ω)** تصف **الحد الأدنى** لمعدل نمو الخوارزمية. إنها تخبرنا بأن أداء الخوارزمية **لن يكون أفضل من** أو أسرع من معدل نمو معين، بغض النظر عن طبيعة المدخلات. بعبارة أخرى، هي تصف **الحالة الأفضل (Best-Case)** للخوارزمية.

نقول إن دالة التعقيد f(n) هي Ω(g(n)) إذا كانت f(n) تنمو **بشكل أسرع من أو بنفس سرعة** g(n) لقيم n الكبيرة.

مثال توضيحي: خوارزمية البحث الخطي

مثال لخوارزمية بسيطة تبحث عن عنصر في مصفوفة غير مرتبة حجمها n عن طريق المرور عليها من البداية إلى النهاية.

- **الحالة الأسوأ :(Big-O)** هي أن يكون العنصر في آخر مكان أو غير موجود أصلاً. في هذه الحالة، سنقوم بـ n عملية مقارنة. إذن، تعقيد الحالة الأسوأ هو O(n).
- **الحالة الأفضل :(Omega)** ما هو أفضل سيناريو ممكن؟ هو أن نجد العنصر الذي نبحث عنه في **الموقع الأول** مباشرة. في هذه الحالة، سنقوم بعملية مقارنة واحدة فقط.
  - عدد العمليات هنا هو 1 رقم ثابت .
  - لذلك، نقول إن تعقيد الحالة الأفضل لهذه الخوارزمية هو **Ω(1)** يُنطق"Omega of 1" .

هذا يعني أن هذه الخوارزمية، في أفضل أحوالها، ستستغرق وقتاً ثابتاً، ولن تكون أسرع من ذلك أبداً.

لماذا نستخدمOmega ؟

صيغة Omega مفيدة لإثبات أن خوارزمية معينة لا يمكن أن تكون أسرع من حد معين. على سبيل المثال، أي خوارزمية تقوم بترتيب مصفوفة عن طريق المقارنة بين العناصر يجب أن تنظر إلى كل عنصر مرة واحدة على الأقل. لذلك، لا يمكن لأي خوارزمية ترتيب أن يكون تعقيدها أفضل من .Ω(n)

.2صيغة - Theta (Θ) الحد المحكم(Tight Bound)

ما هي صيغة Theta ؟

صيغة **Theta (Θ)**هي الأقوى والأكثر دقة من بين الصيغ الثلاث. نستخدمها عندما يكون أداء الخوارزمية محصوراً **بين نفس الحد الأعلى والحد الأدنى** .بعبارة أخرى، نستخدم Theta عندما يكون أداء الحالة الأسوأ (Big-O) والحالة الأفضل (Omega) متماثلاً من حيث معدل النمو.

نقول إن دالة التعقيد f(n)هي Θ(g(n)) إذا كانت f(n) هي O(g(n)) و Ω(g(n))في نفس الوقت.

هذا يعني أن g(n) تصف أداء الخوارزمية بشكل دقيق ومحكم، لا هو مجرد حد أعلى ولا مجرد حد أدنى.

مثال توضيحي (1): خوارزمية جمع عناصر مصفوفة

لنرجع إلى مثال جمع كل عناصر مصفوفة حجمها .n

Python

```
def sum_array(numbers):
  total = 0
  for number in numbers:
    total += number
  return total
```

- **تحليل الأداء :**هل يختلف عدد العمليات باختلاف قيم الأرقام داخل المصفوفة؟ لا. سواء كانت الأرقام صغيرة أو كبيرة، مرتبة أو غير مرتبة، ستقوم الحلقة التكرارية بالمرور على **كل العناصر** n مرة بالضبط.
- **الحالة الأسوأ :(Big-O)** تحتاج إلى n عملية. إذن هي .O(n)
- **الحالة الأفضل :(Omega)** تحتاج أيضاً إلى nعملية. إذن هي .Ω(n)
- **الاستنتاج :**بما أن الحد الأعلى والحد الأدنى متماثلان(n) ، يمكننا استخدام صيغة Theta لوصف الأداء بشكل دقيق.
- لذلك، نقول إن تعقيد هذه الخوارزمية هو) **Θ(n)** يُنطق ("Theta of n")

.

- **مثال توضيحي (2): هل يمكننا استخدام Theta للبحث الخطي؟**

في مثال البحث الخطي السابق:

- الحالة الأسوأ كانت .O(n)
- الحالة الأفضل كانت .Ω(1)

بما أن الحد الأعلى (n) والحد الأدنى (1) **غير متماثلين**، فلا يمكننا استخدام صيغة Theta لوصف أداء هذه الخوارزمية بشكل عام.

تجيب على سؤال	المعنى	الاسم	الصيغة
ما هو أسوأ أداء ممكن للخوارزمية؟	**الحد الأعلى** (Upper Bound)	Big-O	**O**
ما هو أفضل أداء ممكن للخوارزمية؟	**الحد الأدنى** (Lower Bound)	Omega	**Ω**
ما هو الأداء الدقيق والمحصور للخوارزمية؟	**الحد المحكم** (Tight Bound)	Theta	**Θ**

في عالم البرمجة، نستخدم Big-O كثيرًا في الوظائف والمشاريع لأنها تساعدنا على معرفة **أسوأ أداء ممكن** للخوارزمية، وهذا مهم لتجنب المفاجآت عندما يكون عدد البيانات كبيرًا جدًا اما **Omega (Ω)**و **Theta (Θ)** فهو يمنحك **رؤية أعمق وتحليلًا أدق** لأداء الخوارزميات .

أمثلة إضافية على تبسيط دوال التعقيد باستخدامBig-O

الهدف من هذه الأمثلة هو تطبيق القاعدتين الذهبيتين لتبسيط:Big-O

1. **الاحتفاظ بالحد الأعلى فقط :(Drop lower-order terms)** نهتم فقط بالحد الذي ينمو بشكل أسرع.
2. **تجاهل المعاملات الثابتة :(Drop constant factors)** نتجاهل الأرقام المضروبة في الحد الأعلى.

المثال الأول: دالة تربيعية بسيطة

- **الدالة**$f(n) = 8n^2 + 3n + 4$:
- **الشرح والخطوات:**

1. **تحديد الحد الأعلى :**ننظر إلى حدود الدالة$8n^2$:، $3n$، و 4. عند قيم nالكبيرة، فإن الحد n^2ينمو بشكل أسرع بكثير من الحد nأو الثابت 4. إذن، الحد المهيمن هو $8n^2$.
2. **تجاهل الحدود الأقل أهمية :**نتجاهل $3n$و 4. يتبقى لدينا $8n^2$.
3. **تجاهل المعامل الثابت :**نتجاهل الرقم المضروب في الحد الأعلى، وهو 8.

- **النتيجة النهائية :**تعقيد Big-O هو **$O(n^2)$**.

المثال الثاني: دالة تتضمن لوغاريتم

- **الدالة**$f(n) = 20n + 4n \log(n) + 15$:
- **الشرح والخطوات:**

1. **تحديد الحد الأعلى :**ننظر إلى الحدود$20n$:، $4n \log(n)$، و 15. هنا المقارنة تكون بين nو $n \log(n)$.

1. الحد $\log(n)$ ينمو ببطء، ولكنه لا يزال ينمو. عندما نضربه في n، فإن الحد $n \log(n)$ينمو بشكل أسرع من n لوحدها.
2. على سبيل المثال، إذا كانت $n = 1000$، فإن nتساوي 1000، بينما $n \log(n)$تساوي $1000 * \log(1000)$، وهو رقم أكبر بكثير.

3. إذن، الحد المهيمن هو .4n log(n)

2. **تجاهل الحدود الأقل أهمية :**نتجاهل 20n و .15يتبقى لدينا .4n log(n)

3. **تجاهل المعامل الثابت :**نتجاهل الرقم .4

- **النتيجة النهائية :**تعقيد Big-O هو .**O(n log n)**

المثال الثالث: دالة ثابتة لا تعتمد على n

- **الدالة**f(n) = 500 :
- **الشرح والخطوات:**

1. **تحديد الحد الأعلى :**هذه الدالة لا تحتوي على المتغير .nقيمتها ثابتة دائماً وهي 500، بغض النظر عن حجم المدخلات.
2. **التحليل :**عندما لا يعتمد عدد العمليات على n، فإننا نصف هذا التعقيد بأنه "ثابت."

- **النتيجة النهائية :**تعقيد Big-O هو .**O(1)**الرقم 1هنا يرمز إلى أي عملية ثابتة، وليس بالضرورة عملية واحدة فقط.

المثال الرابع: دالة أسية (وهي من أسوأ التعقيدات)

- **الدالة**$f(n) = 3 * 2^n + 99n^{10} + 5000$:
- **الشرح والخطوات:**

1. **تحديد الحد الأعلى :**ننظر إلى الحدود$3 * 2^n$:، $99n^{10}$، و .5000 المقارنة الأهم هنا هي بين الدالة الأسية 2^nوالدالة كثيرة الحدود .n^{10}

1. قد يبدو الحد n^{10}كبيراً، ولكن النمو الأسي (2^n) هو أسرع بشكل لا يضاهى.
2. على سبيل المثال، إذا كانت :n = 100
1. n^{10}هو 100^{10}رقم ضخم.
2. 2^nهو 2^{100}رقم أكبر بشكل فلكي
3. الدوال الأسية دائماً تهيمن على أي دالة كثيرة الحدود .(polynomial) إذن، الحد المهيمن هو .$3 * 2^n$

2. **تجاهل الحدود الأقل أهمية :**نتجاهل $99n^{10}$و .5000يتبقى لدينا 3 * 2^n.

3. **تجاهل المعامل الثابت :**نتجاهل الرقم 3.

- **النتيجة النهائية :**تعقيد Big-O هو .**$O(2^n)$**

الجدول :

الدالة الأصلية f(n)	الحد المهيمن	تعقيدBig-O
$10n^3 + 5n^2 + 2n$	$10n^3$	**$O(n^3)$**
$n + \log(n)$	n	**$O(n)$**
$7n \log(n) + 300n$	$7n \log(n)$	**$O(n \log n)$**
42	42	**$O(1)$**
$n^2 + 2^n$	2^n	**$O(2^n)$**
$n! + n^{100}$	$n!$	**$O(n!)$**

مثال يوضح العلاقة بين الصيغ الثلاث (O, Ω, Θ) بشكل مثالي عند التعامل مع نوع معين من الدوال (كثيرات الحدود).

دعونا نحلل الدالة $f(n) = 5n^3 + 2n^2 + 1000$ باستخدام كل صيغة من الصيغ التقاربية.

تحليل - Big-O (O) الحد الأعلى

الهدف :إيجاد الحد الأعلى (Upper Bound) الذي يصف أسوأ أداء ممكن أو معدل النمو الأقصى للدالة.

الخطوات:

1. **تحديد الحد الأعلى :(Highest Order Term)** ننظر إلى جميع حدود الدالة $5n^3, 2n^2$: و 1000. الحد الذي ينمو بشكل أسرع كلما زادت قيمة n هو $5n^3$.
2. **تجاهل الحدود الأقل أهمية :**نتجاهل $2n^2$ و 1000 لأن تأثيرهما يصبح ضئيلاً جداً مقارنةً بـ $5n^3$ عند قيم n الكبيرة. يتبقى لدينا $5n^3$.
3. **تجاهل المعامل الثابت :**نتجاهل الرقم 5 المضروب في الحد الأعلى.

النتيجة:

- تعقيد Big-O لهذه الدالة هو **$O(n^3)$**.

المعنى :هذا يعني أن معدل نمو الدالة $f(n)$ **لن يكون أسوأ من**)أو أسرع من) معدل نمو n^3. الدالة n^3 تضع سقفاً أو حداً أعلى لنمو دالتنا.

2.تحليل - Omega (Ω) الحد الأدنى

الهدف :إيجاد الحد الأدنى (Lower Bound) الذي يصف أفضل أداء ممكن أو معدل النمو الأدنى للدالة.

الخطوات:

1. **تحديد الحد المهيمن :(Dominant Term)** مرة أخرى، ننظر إلى جميع حدود الدالة $5n^3, 2n^2$: و 1000. الحد الذي ينمو بشكل أسرع هو $5n^3$.

2. **التحليل :**بما أن $5n^3$هو الجزء الأسرع نمواً في الدالة، فإن الدالة بأكملها ($f(n)$)ستنمو **على الأقل** بنفس سرعة هذا الحد المهيمن. الحدود الأخرى ($2n^2$)و (1000)هي قيم موجبة تضاف إليه، مما يؤكد أن النمو لن يقل عن n^3.

3. **تطبيق قواعد التبسيط :**تماماً كما فيBig-O ، نتجاهل المعاملات الثابتة والحدود الأقل أهمية عند تحديد فئة النمو.

النتيجة:

- تعقيد Omega لهذه الدالة هو **$\Omega(n^3)$**.

المعنى :هذا يعني أن معدل نمو الدالة ($f(n)$**لن يكون أفضل من**)أو أبطأ من) معدل نمو n^3.الدالة n^3تضع أرضية أو حداً أدنى لنمو دالتنا.

3.تحليل - Theta (Θ) الحد المحكم

الهدف :تحديد ما إذا كان يمكن وصف نمو الدالة بشكل دقيق ومحكم، أي أن يكون الحد الأعلى والحد الأدنى من نفس فئة النمو.

الخطوات:

1. **مقارنة Big-O وOmega:**

1. لقد وجدنا أن الحد الأعلى للدالة هو $O(n^3)$.
2. ووجدنا أن الحد الأدنى للدالة هو $\Omega(n^3)$.

2. **التحقق من الشرط :**شرط استخدام Theta هو أن تكون الدالة $f(n)$هي $O(g(n))$و $\Omega(g(n))$في نفس الوقت، وباستخدام نفس الدالة $g(n)$.

3. **الاستنتاج :**بما أن الحد الأعلى ($O(n^3)$) والحد الأدنى ($\Omega(n^3)$) متطابقان، فإن الشرط قد تحقق. هذا يعني أن نمو الدالة $f(n)$محصور بشكل محكم من الأعلى والأسفل بنفس معدل النمو n^3.

النتيجة:

- تعقيد Theta لهذه الدالة هو .**$\Theta(n^3)$**

المعنى :هذا هو الوصف الأكثر دقة للدالة. إنه يخبرنا بأن معدل نمو $f(n)$هو **بالضبط** نفس معدل نمو n^3عند المدخلات الكبيرة.

خلاصة

عندما تكون دالة التعقيد عبارة عن **دالة كثيرات الحدود (Polynomial Function)**مثل $f(n) = 5n^3 + 2n^2 + 1000$، فإن الحد الأعلى (O) والحد الأدنى (Ω) والحد المحكم (Θ) يتفقون جميعاً على نفس فئة النمو، وهي فئة الحد الأعلى في الدالة.

النتيجة	التحليل	الصيغة
$O(n^3)$	تصف الحد الأعلى للنمو	**Big-O (O)**
$\Omega(n^3)$	تصف الحد الأدنى للنمو	**Omega (Ω)**
$\Theta(n^3)$	تصف النمو المحكم والدقيق	**Theta (Θ)**

عندما تكون دالة التعقيد عبارة عن مجموع من الحدود الموجبة (كما في كل الأمثلة أدناه)، فإن الحد الأعلى نمواً يُشكل العامل الرئيسي في سلوك الدالة من الأعلى ومن الأسفل.

الدالة الأصلية f(n)	العامل الرئيسي	تعقيد Big-O (O)	تعقيد Omega (Ω)	تعقيد Theta (Θ)
$10n^3 + 5n^2 + 2n$	$10n^3$	$O(n^3)$	$\Omega(n^3)$	$\Theta(n^3)$
$n + \log(n)$	n	$O(n)$	$\Omega(n)$	$\Theta(n)$
$7n \log(n) + 300n$	$7n \log(n)$	$O(n \log n)$	$\Omega(n \log n)$	$\Theta(n \log n)$
42	42	$O(1)$	$\Omega(1)$	$\Theta(1)$
$n^2 + 2^n$	2^n	$O(2^n)$	$\Omega(2^n)$	$\Theta(2^n)$
$n! + n^{100}$	$n!$	$O(n!)$	$\Omega(n!)$	$\Theta(n!)$

لماذا تتطابق الصيغ الثلاث في هذه الأمثلة؟

دعنا نأخذ المثال الأول $f(n) = 10n^3 + 5n^2 + 2n$:

1. شرح Big-O: نحن نبحث عن "سقف" أو حد أعلى. بما أن n^3 هو أسرع الحدود نمواً، فمن المؤكد أن الدالة بأكملها لن تنمو أسرع من n^3. لذلك، $O(n^3)$ هو وصف صحيح للحد الأعلى.
2. شرح Omega (Ω): نحن نبحث عن "أرضية" أو حد أدنى. بما أن $10n^3$ هو جزء من الدالة، والدالة تحتوي أيضاً على حدود أخرى موجبة ($5n^2$ و $2n$)، فمن المؤكد أن الدالة بأكملها ستنمو على الأقل بنفس سرعة n^3. لا يمكن أن تنمو بشكل أبطأ من حدها المهيمن. لذلك، $\Omega(n^3)$ هو وصف صحيح للحد الأدنى.

3. شرح :Theta (Θ) بما أن "السقف ($O(n^3)$) "و"الأرضية ($\Omega(n^3)$) "لهما نفس معدل النمو، فإننا نقول إن نمو الدالة "محصور بشكل محكم tightly) " (bounded بواسطة n^3. هذا يسمح لنا باستخدام الوصف الأكثر دقة وهو $\Theta(n^3)$.

أمثلة على Big-O, Omega, وTheta

لفهم الفروقات الجوهرية بين الصيغ التقاربية الثلاث، دعونا نحلل خوارزميتين معروفتين ونحدد تعقيد كل منهما باستخدام O, Ω, وΘ.

المثال الأول: البحث الخطي(Linear Search)

المشكلة :ابحث عن عنصر معين (key) في مصفوفة غير مرتبة حجمها n.

الخوارزمية (كود بايثون):

Python

```
def linear_search(data_list, key):
    تبحث عن عنصر في قائمة وتعيد موقعه إذا وجد، وإلا تعيد -1.
    for i in range(len(data_list)): # الحلقة تدور n مرة في أسوأ الأحوال
        if data_list[i] == key:
            return i  # تم العثور على العنصر
    return -1 # العنصر غير موجود
```

التحليل:

1. **تحليل الحالة الأفضل:(Best Case)**

1. **السيناريو :**أن يكون العنصر الذي نبحث عنه (key) هو **أول عنصر** في المصفوفة.(data_list[0])

2. **العمليات :**ستنفذ الحلقة التكرارية مرة واحدة فقط، وستتم عملية مقارنة واحدة (if) قبل أن تعود الدالة بالنتيجة.

3. **الاستنتاج :**عدد العمليات ثابت (1) ولا يعتمد على حجم المصفوفة .n

4. **صيغة :(Ω) Omega** بما أن أفضل أداء ممكن هو وقت ثابت، فإن الحد الأدنى هو .**Ω(1)**

2. **تحليل الحالة الأسوأ:(Worst Case)**

0. **السيناريو :**أن يكون العنصر الذي نبحث عنه (key) هو **آخر عنصر** في المصفوفة، أو أن يكون **غير موجود** على الإطلاق.

1. **العمليات :**في كلتا الحالتين، ستضطر الخوارزمية للمرور على **جميع** عناصر المصفوفة. ستدور الحلقة nمن المرات، وستنفذ nعملية مقارنة.

2. **الاستنتاج :**عدد العمليات يتناسب طردياً مع .n

3. **صيغة :(O) Big-O** بما أن أسوأ أداء ممكن هو وقت خطي، فإن الحد الأعلى هو .**O(n)**

3. **تحليل:(Θ) Theta**

0. **السؤال :**هل يمكننا استخدام صيغة Theta لوصف هذه الخوارزمية؟

1. **الإجابة :**لا. لكي نستخدمTheta ، يجب أن يكون معدل نمو الحالة الأفضل (Ω) هو نفسه معدل نمو الحالة الأسوأ.(O)

2. هنا، لدينا Ω(1)و .O(n)بما أن 1لا تساوي n، فلا يوجد حد محكم (Tight Bound)يصف كل الحالات. أداء الخوارزمية يتراوح بين وقت ثابت ووقت خطي.

ملخص الحل للمثال الأول:

- **Big-O:** O(n)
- **Omega (Ω):** Ω(1)
- **Theta (Θ):**لا يمكن تحديدها.(Not applicable)

المثال الثاني: إيجاد أكبر عنصر في مصفوفة

المشكلة :أوجد أكبر عنصر في مصفوفة غير مرتبة حجمها n.

الخوارزمية (كود بايثون):

Python

```
def find_maximum(data_list):
    تبحث عن أكبر عنصر في قائمة وتعيده.
    if not data_list: # حالة خاصة إذا كانت القائمة فارغة
        return None
    max_value = data_list[0] # عملية واحدة: نفترض أن الأول هو الأكبر
    # الحلقة تدور n-1 مرة
    for i in range(1, len(data_list)):
        if data_list[i] > max_value:
            max_value = data_list[i] # عملية تحديث
    return max_value
```

التحليل:

1. **تحليل الحالة الأفضل:(Best Case)**

1. **السيناريو :**أن تكون المصفوفة مرتبة تنازلياً، وبالتالي يكون **أول عنصر** هو الأكبر بالفعل.

2. **العمليات :**هل هذا يغير عدد المقارنات؟ لا. حتى لو كان أول عنصر هو الأكبر، **يجب على الخوارزمية أن تمر على جميع العناصر الأخرى** لتتأكد من عدم وجود عنصر أكبر منه. ستدور الحلقة n-1مرة وستقوم بـ n-1عملية مقارنة.

3. **الاستنتاج :**عدد العمليات يتناسب طردياً مع n.

4. **صيغة :(Ω) Omega** أفضل أداء ممكن لا يزال يتطلب المرور على كل العناصر، إذن الحد الأدنى هو .**Ω(n)**

2. **تحليل الحالة الأسوأ:(Worst Case)**

0. **السيناريو :**أن تكون المصفوفة مرتبة تصاعدياً، وبالتالي يكون **آخر عنصر** هو الأكبر.

1. **العمليات :**في هذه الحالة، ستمر الخوارزمية على جميع العناصر-n) 1مقارنة)، وفي كل مقارنة ستجد أن العنصر الحالي أكبر، فتقوم بتحديث قيمة .max_valueلكن عدد المقارنات الرئيسي لا يزال -n 1.

2. **الاستنتاج :**عدد العمليات يتناسب طردياً مع .n

3. **صيغة :(O) Big-O** أسوأ أداء ممكن يتطلب المرور على كل العناصر، إذن الحد الأعلى هو .**O(n)**

3. **تحليل:(Θ) Theta**

0. **السؤال :**هل يمكننا استخدام صيغة Theta لوصف هذه الخوارزمية؟

1. **الإجابة :**نعم. لأن معدل نمو الحالة الأفضل هو Ω(n)ومعدل نمو الحالة الأسوأ هو .O(n)

2. بما أن الحدين الأعلى والأدنى متطابقان (كلاهما ينمو بشكل خطي)، فإننا نصف أداء هذه الخوارزمية بشكل دقيق باستخدام.Theta

3. **الاستنتاج :**التعقيد المحكم لهذه الخوارزمية هو .**Θ(n)**

ملخص الحل للمثال الثاني:

- **Big-O:** O(n)
- **Omega (Ω):** Ω(n)
- **Theta (Θ):** Θ(n)

.